

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO1: *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks: 25

Annexure A

APPLICATION BASED QUESTIONS

Code of Conduct:

*I **A.G.MANOJ/192312179** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: A.G.MANOJ

SET NO 03:

11. In Shop X, there are 20 bottles of pure oil and 30 bottles of adulterated oil, while in Shop Y, there are 40 bottles of pure oil and 20 bottles of adulterated oil. A bottle is selected at random from one of the shops, with Shop Y being twice as likely to be chosen as Shop X. The selected bottle is found to be adulterated.

PYTHON CODE:

```
ASSESSMENTS / 1st.py / ...
1  from fractions import Fraction as F
2
3  P_X = F(1, 3)
4  P_Y = F(2, 3)
5
6  P_A_given_X = F(30, 50)
7  P_A_given_Y = F(20, 60)
8
9  P_A = P_X * P_A_given_X + P_Y * P_A_given_Y
10
11 P_X_given_A = (P_X * P_A_given_X) / P_A
12 P_Y_given_A = (P_Y * P_A_given_Y) / P_A
13
14 print(f"P(Shop X | Adulterated) = {P_X_given_A} = {float(P_X_given_A):.4f}")
15 print(f"P(Shop Y | Adulterated) = {P_Y_given_A} = {float(P_Y_given_A):.4f}")
```

OUTPUT:

```
> & C:\Users\arlag\AppData\Local\Programs\Python\Python310-32\python.exe c:/Users/arlag/OneDrive/Desktop/ASSESSMENTS/1ST.py
P(Shop X | Adulterated) = 9/19 = 0.4737
P(Shop Y | Adulterated) = 10/19 = 0.5263
```

12. A warehouse has two sections. Section A contains 15 defective items and 35 non-defective items, while Section B contains 25 defective items and 25 non-defective items. One section is chosen at random, but Section A is three times as likely to be chosen as Section B. An item selected from the chosen section is found to be defective.

PYTHON CODE:

```

1  from fractions import Fraction as F
2
3  P_A = F(3, 4)
4  P_B = F(1, 4)
5
6  P_D_given_A = F(15, 50)
7  P_D_given_B = F(25, 50)
8
9  P_D = P_A * P_D_given_A + P_B * P_D_given_B
10
11 P_A_given_D = (P_A * P_D_given_A) / P_D
12 P_B_given_D = (P_B * P_D_given_B) / P_D
13
14 print(f"P(Section A | Defective) = {P_A_given_D} = {float(P_A_given_D):.4f}")
15 print(f"P(Section B | Defective) = {P_B_given_D} = {float(P_B_given_D):.4f}")

```

OUTPUT:

```

> & C:\Users\arlag\AppData\Local\Microsoft\Windows\Apps\python.exe c:/Users/arlag/OneDrive/Desktop/ASSESSMENTS/2nd.py
P(Section A | Defective) = 9/14 = 0.6429
P(Section B | Defective) = 5/14 = 0.3571
PS C:\Users\arlag\OneDrive\Desktop>

```

13. In a binary classification problem using a deep learning model with a K-Nearest Neighbor classifier, the model produces True Positive (TP) = 120 and True Negative (TN) = 150. The dataset contains 1000 samples per class, and 20% of the data is used for testing. (5 Marks)
Find the following: Accuracy, Precision, Sensitivity (Recall), Specificity.

PYTHON CODE:

```

ASSESSMENTS > 3rd.py > ...
1  from fractions import Fraction as F
2
3  TP = 120
4  TN = 150
5
6  pos_test = 200
7  neg_test = 200
8
9  FN = pos_test - TP
10 FP = neg_test - TN
11
12 total = TP + TN + FP + FN
13
14 accuracy = F(TP + TN, total)
15 precision = F(TP, TP + FP)
16 sensitivity = F(TP, TP + FN)
17 specificity = F(TN, TN + FP)
18
19 print(f"Accuracy    = {accuracy} = {float(accuracy):.4f}")
20 print(f"Precision   = {precision} = {float(precision):.4f}")
21 print(f"Sensitivity = {sensitivity} = {float(sensitivity):.4f}")
22 print(f"Specificity = {specificity} = {float(specificity):.4f}")

```

OUTPUT:

```

> & C:\Users\arlag\AppData\Local\Programs\Python\Python310-32\pytho
n.exe c:/Users/arlag/OneDrive/Desktop/ASSESSMENTS/3rd.py
Accuracy    = 27/40 = 0.6750
Precision   = 12/17 = 0.7059
Sensitivity = 3/5 = 0.6000
Specificity = 3/4 = 0.7500
PS C:\Users\arlag\OneDrive\Desktop>

```

14. A medical image classification system using a deep learning framework combined with K-Nearest Neighbor (KNN) yields $TP = 180$ and $TN = 160$. Each class contains 1200 samples, and 25% of the dataset is used as test data. Calculate: Accuracy Precision Sensitivity Specificity.

PYTHON CODE:

```

1  from fractions import Fraction as F
2
3  TP = 180
4  TN = 160
5
6  pos_test = 300
7  neg_test = 300
8
9  FN = pos_test - TP
10 FP = neg_test - TN
11
12 total = TP + TN + FP + FN
13
14 accuracy = F(TP + TN, total)
15 precision = F(TP, TP + FP)
16 sensitivity = F(TP, TP + FN)
17 specificity = F(TN, TN + FP)
18
19 print(f"Accuracy    = {accuracy} = {float(accuracy):.4f}")
20 print(f"Precision   = {precision} = {float(precision):.4f}")
21 print(f"Sensitivity = {sensitivity} = {float(sensitivity):.4f}")
22 print(f"Specificity = {specificity} = {float(specificity):.4f}")

```

OUTPUT:

```

> & C:\Users\arlag\AppData\Local\Programs\Python\Python310-32
n.exe c:/Users/arlag/OneDrive/Desktop/ASSESSMENTS/4th.py
Accuracy    = 17/30 = 0.5667
Precision   = 9/16 = 0.5625
Sensitivity = 3/5 = 0.6000
Specificity = 8/15 = 0.5333
PS C:\Users\arlag\OneDrive\Desktop>

```

15. A deep learning model is trained for a classification task, and the following loss values are observed: (5 Marks)

- i. Epoch 1: Training Loss = 2.8, Validation Loss = 2.6
- ii. Epoch 2: Training Loss = 2.4, Validation Loss = 2.2
- iii. Epoch 3: Training Loss = 2.1, Validation Loss = 1.9
- iv. Epoch 4: Training Loss = 1.9, Validation Loss = 1.8
- v. Epoch 5: Training Loss = 1.7, Validation Loss = 1.9
- vi. Epoch 6: Training Loss = 1.5, Validation Loss = 2.1
- vii. Epoch 7: Training Loss = 1.3, Validation Loss = 2.4
- viii. Epoch 8: Training Loss = 1.2, Validation Loss = 2.7

Answer the following:

- a. At which epoch does the model achieve the best generalization performance?
- b. At which point does overfitting begin? Justify your answer.
- c. Suggest two techniques to reduce overfitting and briefly explain how they help.

PYTHON CODE:

```

A Welcome > 5th.py > ...
1 train_loss = [2.8, 2.4, 2.1, 1.9, 1.7, 1.5, 1.3, 1.2]
2 val_loss    = [2.6, 2.2, 1.9, 1.8, 1.9, 2.1, 2.4, 2.7]
3 epochs      = list(range(1, 9))
4
5 best_epoch = epochs[val_loss.index(min(val_loss))]
6 print("Best epoch (min validation loss):", best_epoch, "-> Val Loss =", min(val_loss))
7
8 overfit_start = None
9 for i in range(1, len(val_loss)):
10     if val_loss[i] > val_loss[i-1] and train_loss[i] < train_loss[i-1]:
11         overfit_start = epochs[i]
12         break
13 print("Overfitting begins at Epoch:", overfit_start)

```

OUTPUT:

```

> & C:\Users\arlag\AppData\Local\Programs\Python\Python310\python.exe c:/Users/arlag/OneDrive/Desktop/ASSESSMENTS/5th.py
Best epoch (min validation loss): 4 -> Val Loss = 1.8
Overfitting begins at Epoch: 5
PS C:\Users\arlag\OneDrive\Desktop>

```